
Génie Logiciel

CER | Multi-Tache

Lundi 09 février 2026

TABLE DES MATIÈRES

Étude de cas : Multi-tâches	2
AAV	2
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	4
Livrables	4
Pistes de solutions	4
Plan d'actions	4
Démarche : Multi-tâches	5
Définition: Mots clés	5
Corbeille Gestion des processus	6
Workshop : Multi-tâches	13
Analyse et résolution : Multi-tâches	22
Conclusion	22
Capitalisation	23
Ressources	24

ÉTUDE DE CAS : MULTI-TÂCHES

AAV

- 3 Décrire parfaitement les deux notions
- 3 Décrire les relations interprocessus (permutation de contexte)
- 1 Enumérer les algorithmes les plus importants
- 3 Coder un des algorithmes de façon simple (« hors contexte » d'un SE)
- 3 Coder un programme en C# pour gérer des processus avec le framework .NET

MOTS CLÉS

- Monoprogrammation
- Multiprogrammation
- Temps partagé
- Préemptif
- PID
- Pointeur Stack
- Processus
- Gestion de processus
- Tâche asynchrone
- Conflits
- Temps processeur
- Ressources

CONTEXTE

Le client demande à Franck une amélioration de son application afin de gérer plusieurs processus à partir d'un seul processus

PROBLÉMATIQUE

Comment gérer efficacement plusieurs processus à partir d'un seul processus, en assurant une utilisation optimale des ressources et une performance adéquate de l'application?

CONTRAINTES

- Un seul processus
- Traiter plusieurs tâches simultanément
- Minimiser les conflits

GÉNÉRALISATION

- Multi-taches
- Processus

LIVRABLES

- Documentation technique de l'implémentation
- Code expliquant la gestion de processus en multi-taches

PISTES DE SOLUTIONS

- Etudier le multi-tâches
- Comprendre le fonctionnement des processus
- Monoprogrammation vs Multiprogrammation
- Etudier les systèmes à temps partagé

PLAN D' ACTIONS

- Etudier les mots clés
- Etudier le multi-tâches
- Comprendre le fonctionnement des processus
- Monoprogrammation vs Multiprogrammation
- Etudier les systèmes à temps partagé
- Implémenter un programme en C# pour gérer des processus avec le framework .NET
- Rédiger la documentation technique de l'implémentation

DÉMARCHE : MULTI-TÂCHES

DÉFINITION : MOTS CLÉS

Monoprogrammation: Exécution d'un seul programme à la fois sur le processeur, sans partage des ressources entre plusieurs processus.

Multiprogrammation: Capacité à exécuter plusieurs programmes simultanément en partageant les ressources du système d'exploitation.

Temps partagé: Technique d'allocation du temps processeur entre plusieurs processus, chacun recevant des tranches de temps (quantum) successives.

Préemptif: Mode d'ordonnancement où le système peut interrompre un processus en cours d'exécution pour en lancer un autre, sans attendre sa terminaison volontaire.

PID: Identifiant unique (Process IDentifier) attribué à chaque processus pour le distinguer des autres au sein du système.

Pointeur Stack: Adresse mémoire pointant vers le sommet de la pile (stack) où sont stockées les variables locales et les adresses de retour.

Processus: Programme en cours d'exécution disposant de son propre contexte (mémoire, registres, PID) et géré par le système d'exploitation.

Gestion de processus: Ensemble des mécanismes permettant au système d'exploitation de créer, ordonnancer, surveiller et terminer les processus.

Tâche asynchrone: Opération exécutée de manière indépendante du flux principal du programme, permettant d'autres traitements en parallèle.

Conflits: Situations où plusieurs processus accèdent simultanément à une ressource partagée, risquant d'incohérence ou de corruption de données.

Temps processeur: Durée réelle d'exécution d'un processus sur le processeur, alloué et géré par l'ordonnanceur du système.

Ressources: Éléments matériels ou logiciels limités (processeur, mémoire, fichiers, entrées/sorties) partagés entre plusieurs processus.

L'ordonnancement de processus: Mécanisme par lequel le système d'exploitation décide quel processus doit être exécuté à un moment donné, en fonction de critères tels que la priorité, le temps d'attente, etc.

CORBEILLE GESTION DES PROCESSUS

Question: 1.1 Quelle est la différence entre les systèmes monoprogrammation et multiprogrammation ? Quelle est la différence entre les systèmes mono et multi utilisateurs ? Pouvez-vous faire un tableau comparatif ?

Caractéristiques	Monoprogrammation	Multiprogrammation
Nombre de programmes	Un seul à la fois	Plusieurs simultanément
Partage des ressources	Non	Oui
Utilisation du processeur	Basse	Élevée
Contexte switch	Aucun	Fréquent
Temps d'attente	Long	Court
Complexité	Simple	Complexe
Système d'exploitation	Basique	Avancé

Caractéristiques	Mono-utilisateur	Multi-utilisateurs
Nombre d'utilisateurs	Un seul	Plusieurs simultanément
Accès au système	Exclusif	Partagé
Gestion des droits	Minimale	Complexe
Sécurité	Simple	Renforcée
Ressources allouées	Totalité du système	Distribuées entre utilisateurs
Isolation des données	N/A	Nécessaire

Question: 1.2 Est-ce qu'on peut parler « vraiment » de multiprogrammation dans un ordinateur avec un seul processeur ?

Oui, même avec un seul processeur, on peut parler de multiprogrammation grâce à la technique du temps partagé. Le système d'exploitation alloue des tranches de temps (quantum) à chaque processus, permettant ainsi à plusieurs programmes de

s'exécuter de manière quasi simultanée en alternant leur accès au processeur. Cela donne l'illusion que les processus s'exécutent en parallèle, même si en réalité ils sont exécutés séquentiellement sur le même processeur.

Question: 1.3 Quelles sont les tâches principales d'un gestionnaire de processus ?

Les tâches principales d'un gestionnaire de processus sont :

- **Création de processus** : Initialiser un nouveau processus avec son propre contexte d'exécution.
- **Ordonnancement** : Déterminer l'ordre d'exécution des processus en fonction de critères tels que la priorité, le temps d'attente, etc.
- **Synchronisation** : Gérer les accès concurrents aux ressources partagées pour éviter les conflits et assurer l'intégrité des données.
- **Communication interprocessus** : Faciliter l'échange de données et de messages entre les processus.
- **Terminaison** : Gérer la fin d'un processus, libérer les ressources allouées et mettre à jour les structures de données du système.
- **Surveillance** : Suivre l'état des processus en cours d'exécution et fournir des informations sur leur statut.

Question: 1.4 Pouvez-vous donner les caractéristiques des mécanismes de commutation/permutation de contexte de processus et d'ordonnancement de processus ?

Caractéristiques	Context Switch	Ordonnancement
Définition	Sauvegarde et restauration du contexte d'un processus	Sélection du prochain processus à exécuter
Moment	Entre chaque changement de processus	À chaque interruption ou fin de quantum
Opération	Sauvegarde des registres, PC et état	Choix basé sur critères de priorité
Coût	Élevé (ralentissement du système)	Variable selon l'algorithme
Fréquence	Fréquente en multiprogrammation	Fréquente en temps partagé
Impact	Perte de cache, surcharge système	Équilibre entre équité et performance
Objectif	Préserver l'état d'exécution du processus	Optimiser utilisation des ressources

Question: 1.5 Quels types d'algorithmes de « scheduling » connaissez-vous ? Pouvez-vous donner ses avantages et ses inconvénients ?

Algorithme	Avantages	Inconvénients
First-Come, First-Served (FCFS)	Simple à implémenter, équitable pour les processus courts	Peut entraîner un temps d'attente élevé pour les processus longs (effet de convoyage)
Shortest Job Next (SJN)	Minimise le temps d'attente moyen	Difficile à implémenter, peut entraîner une famine pour les processus longs
Round Robin (RR)	Équitable, bon pour les systèmes interactifs	Peut entraîner un temps d'attente élevé si le quantum est mal choisi
Priority Scheduling	Permet de donner la priorité aux processus importants	Peut entraîner une famine pour les processus à faible priorité
Multilevel Queue Scheduling	Permet de gérer différents types de processus avec des politiques d'ordonnancement adaptées	Complexe à implémenter, peut entraîner une famine pour les processus dans les queues inférieures
Multilevel Feedback Queue Scheduling	Permet d'ajuster dynamiquement les priorités des processus en fonction de leur comportement	Complexe à implémenter, peut entraîner une surcharge du système

Question: 1.6 Quelles différences trouvez-vous entre un processus et un programme ?

Un programme est un ensemble de instructions statiques stockées sur un support de stockage (disque dur, mémoire) qui peut être exécuté. Un processus, en revanche, est une instance d'un programme en cours d'exécution, avec son propre contexte d'exécution (mémoire, registres, PID). Un programme peut être considéré comme une recette, tandis qu'un processus est le résultat de la mise en œuvre de cette recette. Un programme peut être exécuté plusieurs fois, créant ainsi plusieurs processus distincts, chacun avec son propre état et ressources.

Question: 1.7 Un quantum est associé à un processus dans un système temps partagé. Est-ce que le processus « peut quitter » l'UC avant la fin de ce quantum ?

Oui, un processus peut quitter l'unité centrale (UC) avant la fin de son quantum dans un système à temps partagé. Cela peut se produire si le processus termine son exécution ou s'il est bloqué en attente d'une ressource (par exemple, une opération d'entrée/sortie). Dans ces cas, le système d'exploitation peut interrompre le processus et libérer l'UC pour permettre à un autre processus de s'exécuter, même si le quantum initial n'est pas encore écoulé.

Question: 1.8 Quels sont les critères souvent utilisés par les politiques d'ordonnancement ?

Les critères souvent utilisés par les politiques d'ordonnancement incluent :

- **Priorité** : Certains processus peuvent être considérés comme plus importants que d'autres et recevoir une priorité plus élevée.
- **Temps d'attente** : Le temps que les processus ont passé en attente peut être pris en compte pour éviter la famine.
- **Temps de service** : La durée estimée d'exécution d'un processus peut influencer son ordonnancement, favorisant les processus courts.
- **Équité** : Assurer que tous les processus ont une chance équitable d'accéder à l'UC, en particulier dans les systèmes interactifs.
- **Utilisation des ressources** : Prendre en compte les ressources nécessaires pour l'exécution d'un processus, comme la mémoire ou les périphériques d'entrée/sortie.

Question: 1.9 Pouvez-vous donner les avantages et inconvénients de mécanismes préemptif et collaboratif ? Pourquoi les SE devraient TOUS avoir un mode préemptif ?

Caractéristiques	Préemptif	Collaboratif
Interruption	Système interrompt le processus	Processus cède volontairement le contrôle
Avantages	Meilleure réactivité, équité garantie, évite la famine	Simple à implémenter, moins de surcharge
Inconvénients	Fréquents changements de contexte, complexité de synchronisation	Risque de famine, moins adapté aux systèmes interactifs
Performance	Peut engendrer une surcharge système	Potentiellement plus efficace
Fiabilité	Un processus bloqué ne paralyse pas le système	Un processus mal conçu peut paralyser le système
Cas d'usage	Systèmes temps réel, multi-utilisateurs	Systèmes simples, processus coopératifs

Tous les systèmes d'exploitation devraient disposer d'un mode préemptif pour garantir une meilleure gestion des ressources, une réactivité accrue, éviter les problèmes de famine et assurer une expérience utilisateur plus fluide et équitable.

Question: 1.10 Qu'est qu'on appelle un processus « léger » ? Quelles sont les différences avec un processus « normal/classique » ?

Un processus « léger », souvent appelé thread, est une unité d'exécution plus petite qu'un processus classique. Les threads partagent le même espace d'adressage et les ressources du processus parent, ce qui les rend plus légers en termes de consommation de mémoire et de temps de création. En revanche, un processus classique a son propre espace d'adressage et ressources, ce qui le rend plus lourd à gérer. Les threads sont généralement utilisés pour des tâches concurrentes au sein d'une même application, tandis que les processus classiques sont utilisés pour des applications distinctes.

Les différences entre un processus léger (thread) et un processus classique incluent :

- **Espace d'adressage** : Les threads partagent le même espace d'adressage, tandis que les processus classiques ont des espaces d'adressage séparés.
- **Ressources** : Les threads partagent les ressources du processus parent, tandis que les processus classiques ont leurs propres ressources.
- **Création et gestion** : Les threads sont plus rapides à créer et à gérer que les processus classiques en raison de leur nature légère.
- **Communication** : La communication entre threads est plus facile et plus rapide que la communication entre processus classiques, qui nécessite des mécanismes d'interprocessus plus complexes.

Question: 1.11 Les processus peuvent communiquer entre eux ?

Oui, les processus peuvent communiquer entre eux à l'aide de mécanismes d'inter-processus (IPC) tels que les pipes, les files de messages, les sémaphores, les mémoires partagées, etc. Ces mécanismes permettent aux processus d'échanger des données et de synchroniser leurs actions, même s'ils sont exécutés dans des espaces d'adressage séparés.

Question: 1.12 Quelles sont les informations minimales pour pouvoir créer un processus ?

Les informations minimales pour créer un processus incluent :

- **Programme à exécuter** : Le chemin vers l'exécutable ou le script à lancer.
- **Arguments** : Les paramètres nécessaires pour l'exécution du programme.
- **Environnement** : Les variables d'environnement nécessaires pour le processus.
- **Permissions** : Les droits d'accès nécessaires pour exécuter le processus.
- **Ressources** : Les ressources nécessaires pour le processus, telles que la mémoire, les fichiers, etc.

- **PID** : Un identifiant unique pour le processus, généralement attribué par le système d'exploitation lors de sa création.
- **Contexte d'exécution** : Les informations sur l'état du processus, telles que les registres, le compteur de programme, etc.

Question: 1.13 Quels états de processus connaissez-vous ?

Les états de processus couramment reconnus sont :

- **Nouveau** : Le processus est en cours de création.
- **Prêt** : Le processus est prêt à être exécuté, mais attend que le processeur soit disponible.
- **En cours d'exécution** : Le processus est actuellement exécuté sur le processeur.
- **Bloqué** : Le processus est en attente d'une ressource ou d'un événement pour continuer son exécution.
- **Terminé** : Le processus a terminé son exécution et attend que le système d'exploitation libère ses ressources.

Question: 1.14 Le SE est un chargeur de programme ?

Non, le système d'exploitation (SE) n'est pas simplement un chargeur de programme. Le SE est un ensemble de logiciels qui gère les ressources matérielles et logicielles d'un ordinateur, fournit des services aux applications et assure la communication entre les utilisateurs et le matériel. Le chargeur de programme est une composante du SE responsable de charger les programmes en mémoire pour leur exécution, mais le SE englobe également d'autres fonctions telles que la gestion des processus, la gestion de la mémoire, la gestion des fichiers, la sécurité, etc.

Question: 1.15 Le SE est-il un ensemble de logiciels ?

Oui, le système d'exploitation (SE) est un ensemble de logiciels qui gère les ressources matérielles et logicielles d'un ordinateur, fournit des services aux applications et assure la communication entre les utilisateurs et le matériel. Le SE comprend des composants tels que le noyau, les pilotes de périphériques, les utilitaires système, les interfaces utilisateur, etc.

Question: 1.16 On peut coder nos propres processus ?

Oui, il est possible de coder nos propres processus en utilisant des langages de programmation qui offrent des bibliothèques ou des API pour la gestion des processus. Par exemple, en C#, on peut utiliser la classe 'System.Diagnostics.Process' pour créer et gérer des processus. En utilisant cette classe, on peut lancer de nouveaux processus, communiquer avec eux, surveiller leur état, etc. De même, d'autres langages comme Python, Java, C++, etc., offrent également des moyens de créer et de gérer des processus.

Question: 1.17 On va pouvoir définir des espaces de mémoires spécifiques pour retrouver nos programmes en cours d'exécution ? (Lien avec prosit précédent)

Oui, chaque processus en cours d'exécution dispose de son propre espace de mémoire, ce qui permet de retrouver et d'isoler les données et les instructions spécifiques à ce processus. Le système d'exploitation gère ces espaces de mémoire pour assurer que les processus ne se chevauchent pas et que les données restent sécurisées. Cela permet également de faciliter la gestion des processus et de garantir que les programmes s'exécutent de manière stable et sécurisée.

Question: 1.18 Il va falloir conserver la valeur du compteur de programme ?

Oui, il est essentiel de conserver la valeur du compteur de programme (Program Counter) lors de la commutation de contexte entre processus. Le compteur de programme indique l'adresse de la prochaine instruction à exécuter pour un processus donné. Lorsqu'un processus est interrompu pour permettre à un autre processus de s'exécuter, le système d'exploitation doit sauvegarder le compteur de programme du processus interrompu afin de pouvoir reprendre son exécution ultérieurement à partir du point où il a été interrompu. Cela garantit que les processus peuvent être exécutés de manière fluide et sans perte d'informations.

Question: 1.19 On va pouvoir stocker toutes les informations dans la « RAM » ?

Oui, les informations nécessaires à l'exécution des processus, telles que les instructions, les données, les registres, etc., sont généralement stockées dans la mémoire vive (RAM) de l'ordinateur. La RAM permet un accès rapide aux données et instructions nécessaires pour l'exécution des processus. Cependant, il est important de noter que la gestion de la mémoire est une tâche complexe pour le système d'exploitation, qui doit s'assurer que les processus disposent de suffisamment de mémoire pour s'exécuter tout en évitant les conflits et en optimisant l'utilisation des ressources.

Question: 2. L'état du processus devra aussi être conservé ?

Oui, l'état du processus doit également être conservé lors de la commutation de contexte. L'état du processus peut inclure des informations telles que les registres CPU, les variables locales, les descripteurs de fichiers ouverts, etc. Le système d'exploitation doit sauvegarder ces informations pour chaque processus afin de pouvoir reprendre son exécution ultérieurement sans perte de données ou d'instructions. Cela permet de garantir que les processus peuvent être exécutés de manière fluide et sans interruption, même lorsqu'ils sont interrompus pour permettre à d'autres processus de s'exécuter.

WORKSHOP : MULTI-TÂCHES

Question 1

- Créer une application console nommée ConsoleHelloWorld.
- Le programme reçoit deux valeurs en paramètre :
 - valeur 1 : name
 - valeur 2 : durée avant fermeture automatique (en milliseconde)
- Le programme affiche et retourne : « Hello name »
- Tester l'exécution du programme à l'aide d'une commande PowerShell.

```
using System;
using System.Diagnostics;
using System.ComponentModel;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            string name = args[0];

            int waitTime = int.Parse(args[1]);

            Console.WriteLine($"Hello, {name}!");

            System.Threading.Thread.Sleep(waitTime * 1000);
        }
    }
}
```

Question 2

Créer un programme C# (que nous allons appeler processus père ou programme principal) qui crée un nouveau processus qui lance « ConsoleHelloWorld ».

```
using System; using System.Diagnostics;
using System.ComponentModel; using System.IO; using System.Reflection;

class Program
{
    static void Main()
    {
        // Construire le chemin absolu depuis le répertoire de l'exé
        string File = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        string targetPath = Path.GetFullPath(Path.Combine(File, @"..\..\..\..\"));

        var processStartInfo = new ProcessStartInfo
        {
            FileName = targetPath,
            Arguments = "ZelPhy 5",
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        };

        using var process = Process.Start(processStartInfo);

        if (process != null)
        {
            string output = process.StandardOutput.ReadToEnd();
            Console.WriteLine(output);
            Thread.Sleep(3000);

            if (!process.HasExited)
            {
                process.Kill();
            }
        }
    }
}
```

Qu'est-ce que le programme principal fait après lancement du processus ?

Le programme principal lance le processus « ConsoleHelloWorld » avec les arguments « ZelPhy 5 ». Ensuite, il lit la sortie standard du processus et l'affiche dans la console. Après cela, il attend pendant 3 secondes (3000 millisecondes) avant de vérifier si le processus est toujours en cours d'exécution. Si le processus n'a pas encore terminé, le programme principal le termine en appelant la méthode 'Kill()'.

Question 3

Lancer les 2 processus du Q2 un après l'autre.

Rajouter un message dans la Console lors du lancement du processus créé en donnant l'information du nom du processus et de son Id. Lancer les 2 processus

Par exemple, afficher dans la console « Processus explorer n° 4567 » est lancé.

```
using System; using System.Diagnostics;
using System.ComponentModel; using System.IO; using System.Reflection;

class Program
{
    static void Main()
    {
        // Construire le chemin absolu depuis le répertoire de l'exe
        string File = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        string targetPath = Path.GetFullPath(Path.Combine(File, @"..\..\..\..\"));

        // Lancer le premier processus
        var processStartInfo1 = new ProcessStartInfo
        {
            FileName = targetPath,
            Arguments = "Alice 3",
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        };

        using var process1 = Process.Start(processStartInfo1);

        if (process1 != null)
        {
            Console.WriteLine($"Processus {process1.ProcessName}");
            Console.WriteLine($"n° {process1.Id} est lancé");

            string output1 = process1.StandardOutput.ReadToEnd();
            Console.WriteLine(output1);

            process1.WaitForExit(); // Attendre la fin du premier processus
        }
    }
}
```

```

// Lancer le deuxième processus
var processStartInfo2 = new ProcessStartInfo
{
    FileName = targetPath,
    Arguments = "Bob 5",
    RedirectStandardOutput = true,
    RedirectStandardError = true,
    UseShellExecute = false,
    CreateNoWindow = true
};

using var process2 = Process.Start(processStartInfo2);

if (process2 != null)
{
    Console.WriteLine($"Processus {process2.ProcessName}");
    Console.WriteLine($"n° {process2.Id} est lancé");

    string output2 = process2.StandardOutput.ReadToEnd();
    Console.WriteLine(output2);

    Thread.Sleep(3000);

    if (!process2.HasExited)
    {
        process2.Kill();
    }
}
}
}

```

Question 4

Vous avez remarqué en Q1 que le processus père peut se terminer même si le processus fils est encore actif.

Modifier le programme Q2 pour que les processus père attendent que l'explorateur des fichiers soit fermé pour fermer le processus père. Encapsuler le lancement et l'attente dans une fonction et rajouter une attente de 2 secondes au processus père avant de continuer son exécution. Utiliser la fonction créée pour appeler « explorer.exe » et « notepad.exe ».

```
using System; using System.Diagnostics;
using System.ComponentModel; using System.IO; using System.Reflection;

namespace HelloWorld
{
    class Program
    {
        static void Main()
        {
            // Lancer explorer.exe
            LancerEtAttendreProcessus("explorer.exe", "");

            // Attendre 2s avant de continuer
            Console.WriteLine("Attente de 2s avant le prochain processus...");
            Thread.Sleep(2000);

            // Lancer notepad.exe
            LancerEtAttendreProcessus("notepad.exe", "");

            Console.WriteLine("Tous les processus sont terminés.");
            Console.WriteLine("Le processus père se termine.");
        }

        static void LancerEtAttendreProcessus(string nomFichier, string arguments)
        {
            var processStartInfo = new ProcessStartInfo
            {
                FileName = nomFichier,
                Arguments = arguments,
                UseShellExecute = false
            };
        }
    }
}
```

```

try
{
    using var process = Process.Start(processStartInfo);

    if (process != null)
    {
        Console.WriteLine($"Processus {process.ProcessName}");
        Console.WriteLine($"n° {process.Id} est lancé");

        // Attendre que le processus se termine
        process.WaitForExit();

        Console.WriteLine($"Processus {process.ProcessName}");
        Console.WriteLine($"n° {process.Id} est terminé");
    }
}
catch (Exception ex)
{
    Console.WriteLine($"{ex.Message}");
}
}
}

```

Qu'est-ce que vous remarquez ? Trouvez-vous une différence entre les 2 processus lancés ? Que pouvons-nous en conclure ?

Lorsque nous lançons « explorer.exe », le processus père attend que l'explorateur de fichiers soit fermé avant de continuer son exécution. En revanche, lorsque nous lançons « notepad.exe », le processus père attend également que Notepad soit fermé avant de continuer. La différence réside dans le fait que l'explorateur de fichiers peut ouvrir plusieurs fenêtres et rester actif même si une fenêtre est fermée, tandis que Notepad est généralement utilisé pour une seule instance. Cela signifie que le processus père peut rester en attente plus longtemps avec l'explorateur de fichiers si plusieurs fenêtres sont ouvertes, tandis qu'avec Notepad, il se terminera dès que la fenêtre sera fermée. Nous pouvons conclure que le comportement du processus père dépend du comportement du processus fils et de la manière dont il gère ses instances et ses fenêtres.

Question 5

Vous allez passer le paramètre nécessaire pour que l'explorateur de fichiers s'ouvre directement sur le répertoire C:\Windows

Créez également un fichier texte et faites-le ouvrir dans Notepad.

```
using System; using System.Diagnostics;
using System.ComponentModel; using System.IO; using System.Reflection;

namespace HelloWorld
{
    class Program
    {
        static void Main()
        {
            // Lancer explorer.exe sur le répertoire C:\Windows
            LancerEtAttendreProcessus("explorer.exe", @"C:\Windows");

            // Attendre 2 secondes avant de continuer
            Console.WriteLine("Attente de 2s avant le prochain processus...");
            Thread.Sleep(2000);

            // Créer un fichier texte
            string cheminFichier = Path.Combine(Path.GetTempPath(), "notepad.txt");
            File.WriteAllText(cheminFichier, "Bonjour !");
            Console.WriteLine($"Fichier créé : {cheminFichier}");

            // Lancer notepad.exe avec le fichier créé
            LancerEtAttendreProcessus("notepad.exe", cheminFichier);

            Console.WriteLine("Tous les processus sont terminés. " +
                             "Le processus père se termine.");
        }

        static void LancerEtAttendreProcessus(string nomFichier, string arguments)
        {
            var processStartInfo = new ProcessStartInfo
            {
                FileName = nomFichier,
                Arguments = arguments,
                UseShellExecute = true
            };
        }
    }
}
```

```

try
{
    using var process = Process.Start(processStartInfo);

    if (process != null)
    {
        Console.WriteLine($"Processus {process.ProcessName} " +
            $"n° {process.Id} est lancé");

        // Attendre que le processus se termine
        process.WaitForExit();

        Console.WriteLine($"Processus {process.ProcessName}" +
            $"n° {process.Id} est terminé");
    }
}
catch (Exception ex)
{
    Console.WriteLine($"{ex.Message}");
}
}
}
}

```

Question 6

Modifier un des programmes précédents pour qu'en utilisant les verbes ci-dessous, le programme ouvre l'explorateur dans le répertoire C:\Windows et ouvre le fichier « C:\Windows\win.ini » dans l'éditeur de texte par défaut (notepad).

Verbes les plus utilisés

- open Ouvrir un exécutable, document ou répertoire. Verbe par défaut
- edit Editer un document
- print Imprimer un document
- explore Explorer un répertoire
- find Lance une recherche dans un répertoire spécifié

ANALYSE ET RÉOLUTION : MULTI-TÂCHES

CONCLUSION

CAPITALISATION

AAV	Commentaire	Statut
Connaître les deux modes d'allocation : contiguë et fragmentée	Les modes d'allocation contiguë et fragmentée sont explicités avec leurs caractéristiques et cas d'usage.	Acquis
Décrire les techniques d'allocation mémoire	Allocation de taille fixe/variable, pagination et segmentation présentées avec schémas et explications.	Acquis
Expliquer les algorithmes d'allocation : First fit, Best fit, Worst fit	Fonctionnement détaillé et comparatif des trois algorithmes avec exemples concrets.	Acquis
Expliquer les algorithmes de remplacement de pages	Algorithmes Random, FIFO, NRU, LRU, LFU décrits avec illustrations et cas d'usage.	Acquis
Implémenter IDisposable et utiliser using	Pattern IDisposable implémenté, usage de l'instruction using démontré avec gestion des ressources.	Acquis
Décrire l'exécution managée et gestion automatique de la mémoire	Processus CLR, garbage collection et Multi-tâches explicités.	Acquis
Comprendre et expliquer les algorithmes de collecte de mémoire	Mécanismes de garbage collection détaillés avec avantages et limitations.	Acquis
Utiliser les délégués et délégués anonymes	Délégués simples, multicast, expressions lambda et closures implémentés avec exemples.	Acquis
Différencier Stack et Heap	Caractéristiques, performances et limites du Stack versus Heap comparées (vitesse, taille).	Acquis
Expliquer les types d'OS et gestion des processus	Monoprogrammation, multiprogrammation, temps partagé et mécanismes préemptif/collaboratif décrits.	Acquis
Décrire les composants d'un processus	PID, Pointeur Stack, État, Program Counter, Registres CPU, Fichiers ouverts explicités.	Acquis

RESSOURCES

- <https://www.geeksforgeeks.org/operating-system-process-scheduling/>
- <https://www.geeksforgeeks.org/inter-process-communication-ipc/>
- <https://www.geeksforgeeks.org/multithreading-in-c-sharp/>